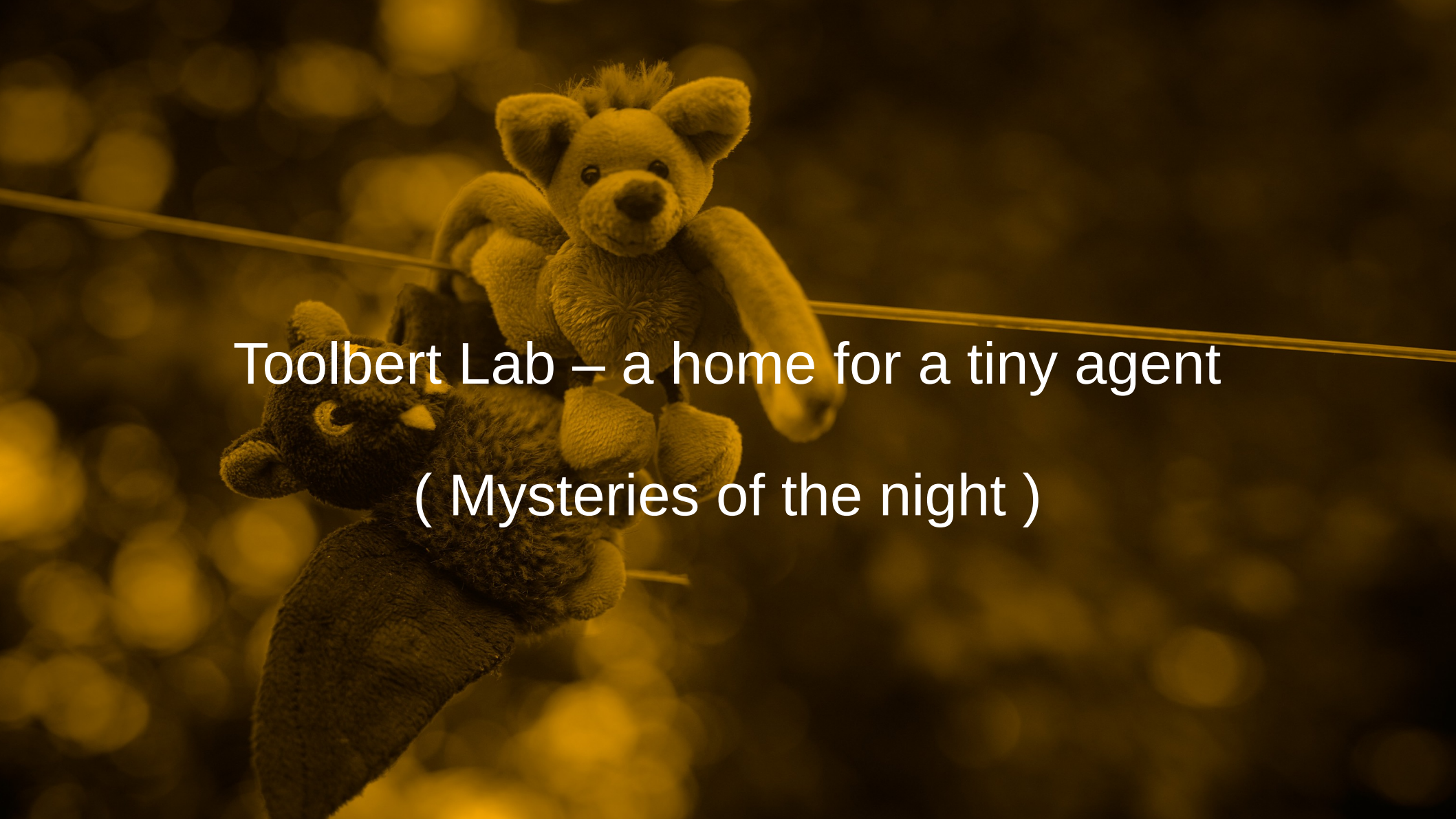
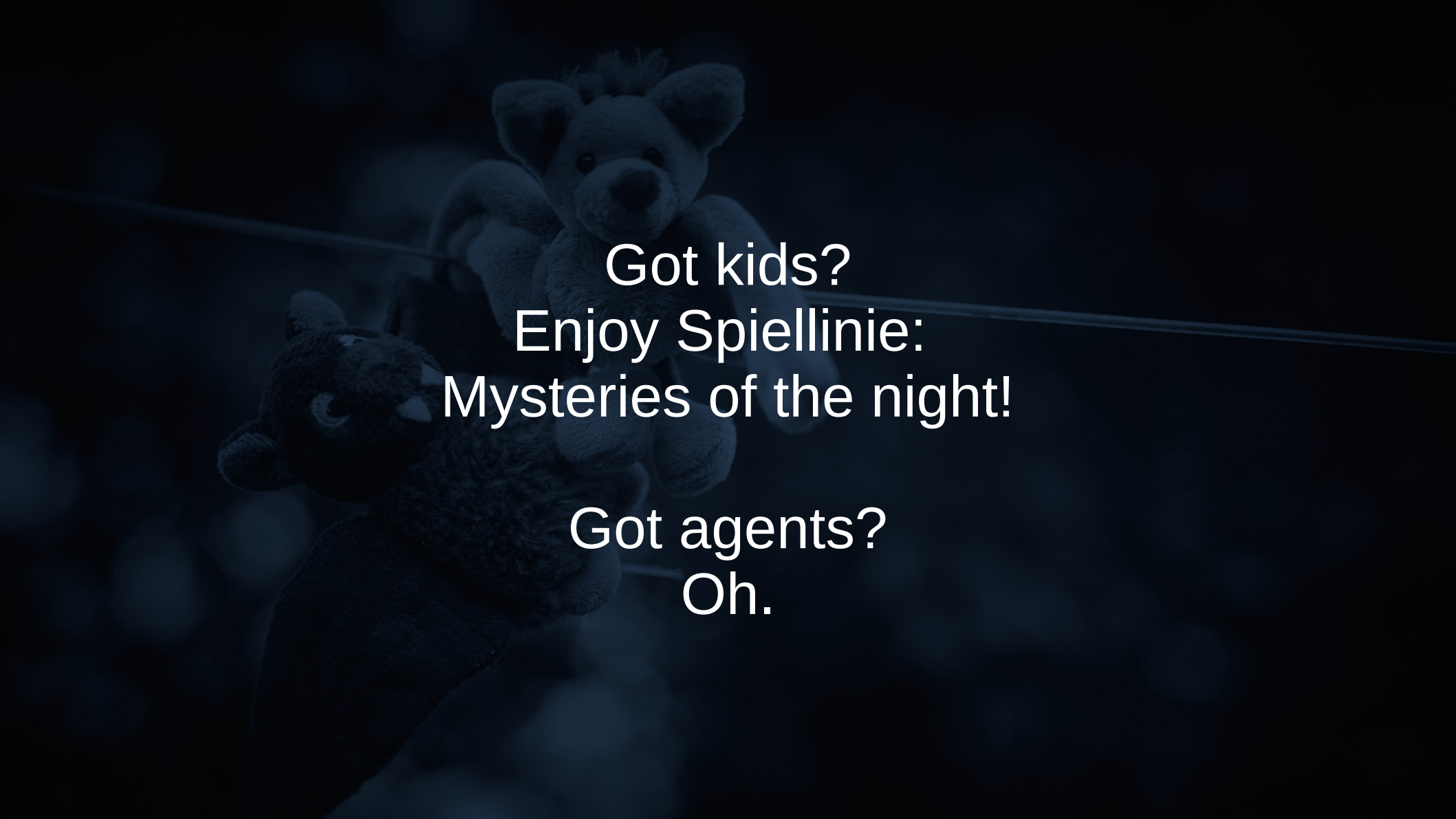


A photograph of two teddy bears on a thin wire. The bear on top is light brown with a tuft of hair on its head. The bear on the bottom is dark brown. The background is dark with many out-of-focus yellow light spots (bokeh).

Toolbert Lab – a home for a tiny agent  
( Mysteries of the night )



Got kids?  
Enjoy Spiellinie:  
Mysteries of the night!

Got agents?  
Oh.

# The speaker says...

Let's wrap up the **OpenCampus course „From LLM to agents“**. The kids can't wait to get to the Kieler Woche, Europe's largest midsummer festival. One of their favourite places is the „Spiellinie“ offering a wide variety of activities including building, creating, painting, reading, listening and jumping.

The motto of 2026 is: Mysteries of the night. This gets us straight to the topic of agents, no?

**Kids sound easy:** they play all day and enjoy ice cream. **Agents sound easy:** just an LLM that reasons and acts (ReAct) and a place to store memories.

**Easy peasy:** all kids and agents need is a **home and supervision**.



# LangGraph, mystery

```
def llm_call(state: dict):  
    [...]
```

```
def tool_node(state: dict):  
    result = []  
    for tool_call in state["messages"][-1].tool_calls:  
        [...]
```

# The speaker says...

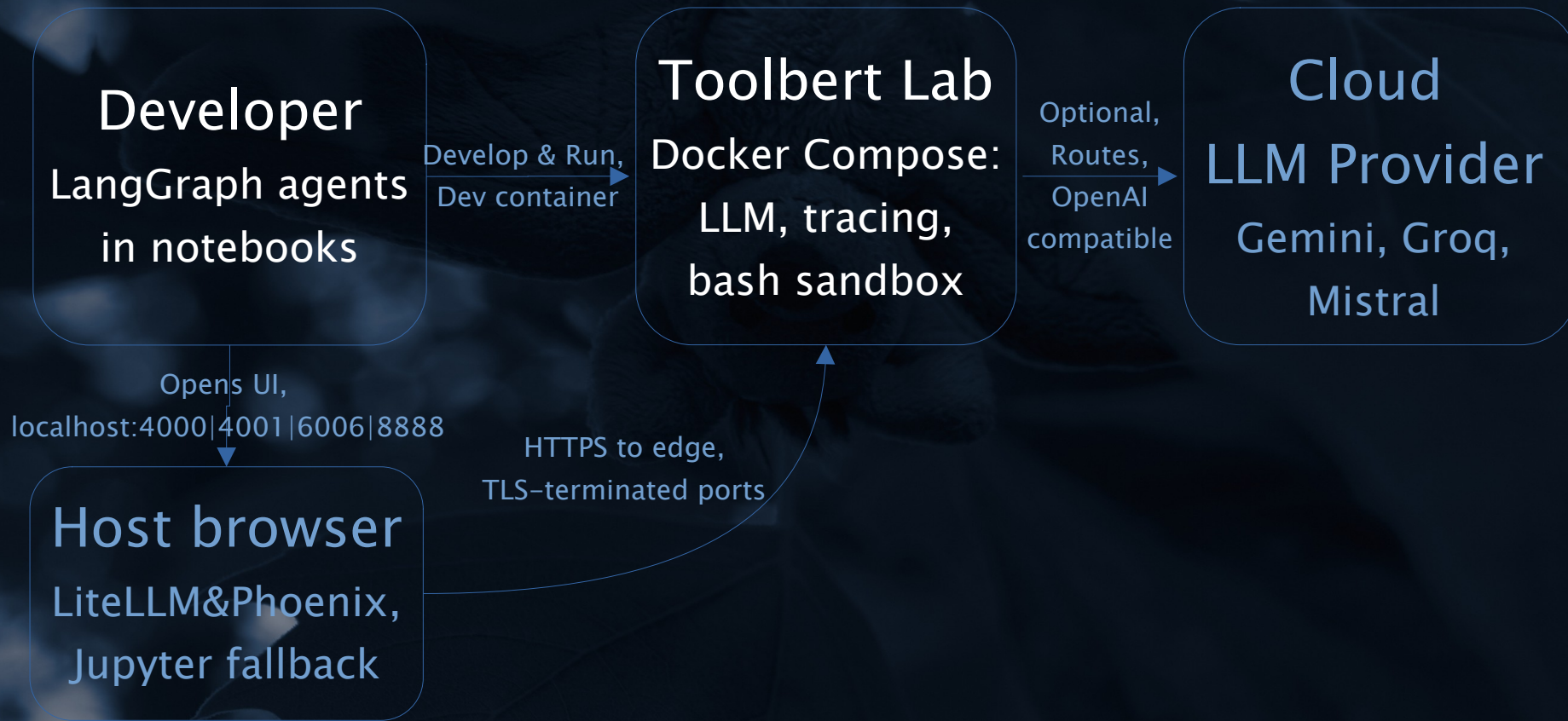
I've opted for the popular LangGraph library to create an agent.

LangGraph has the concept of (a global) state to pass information across the nodes of a graph. This quick start example from the LangGraph manual is certainly intentionally kept very basic.

However, it made me wonder: **where are the contracts? Where are the boundaries? What flows through global state? What can a tool node actually see and do?**

To explore this without getting lost in a real product use case, I picked a deliberately tiny task for the agent, and built a home for agents so I could supervise them.

# A home for agents

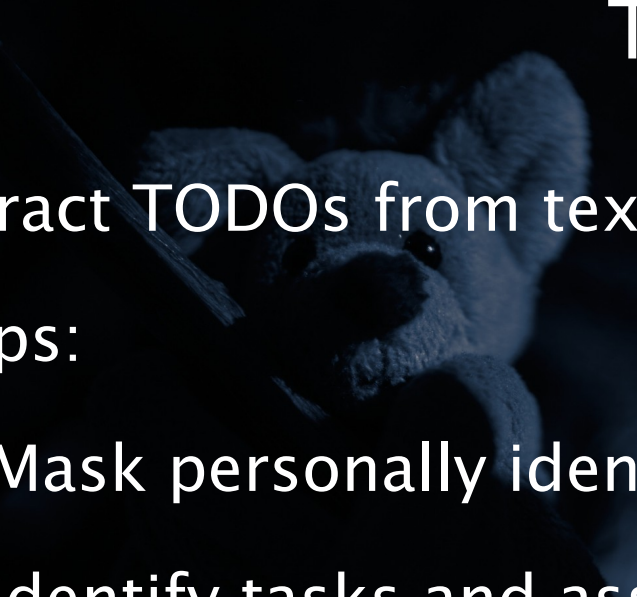


# The speaker says...

I built a local lab as a home for agents in the making:

- Ollama: **run models locally**, local data, controllable environment
- LiteLLM: **OpenAI-compatible gateway, routing, API keys**, budgets, pacing (plugin)
- Phoenix: **tracing, observability**, evals, prompt experiments
- Toxiproxy: **latency**, timeouts, connection resets, chaos testing
- Postgres/pgvector: later RAG and vector store, persistent storage for services
- Caddy: **clean local endpoints**, TLS
- DevContainer: **reproducible development setup**
- Sysbox: **example of an agent sandbox** – that's in the Lab/house; now, the task

# The tiny task



Extract TODOs from text.

Steps:

- Mask personally identifiable information (PII)
- Identify tasks and assignments
- Format TODOs using markup
- Demask



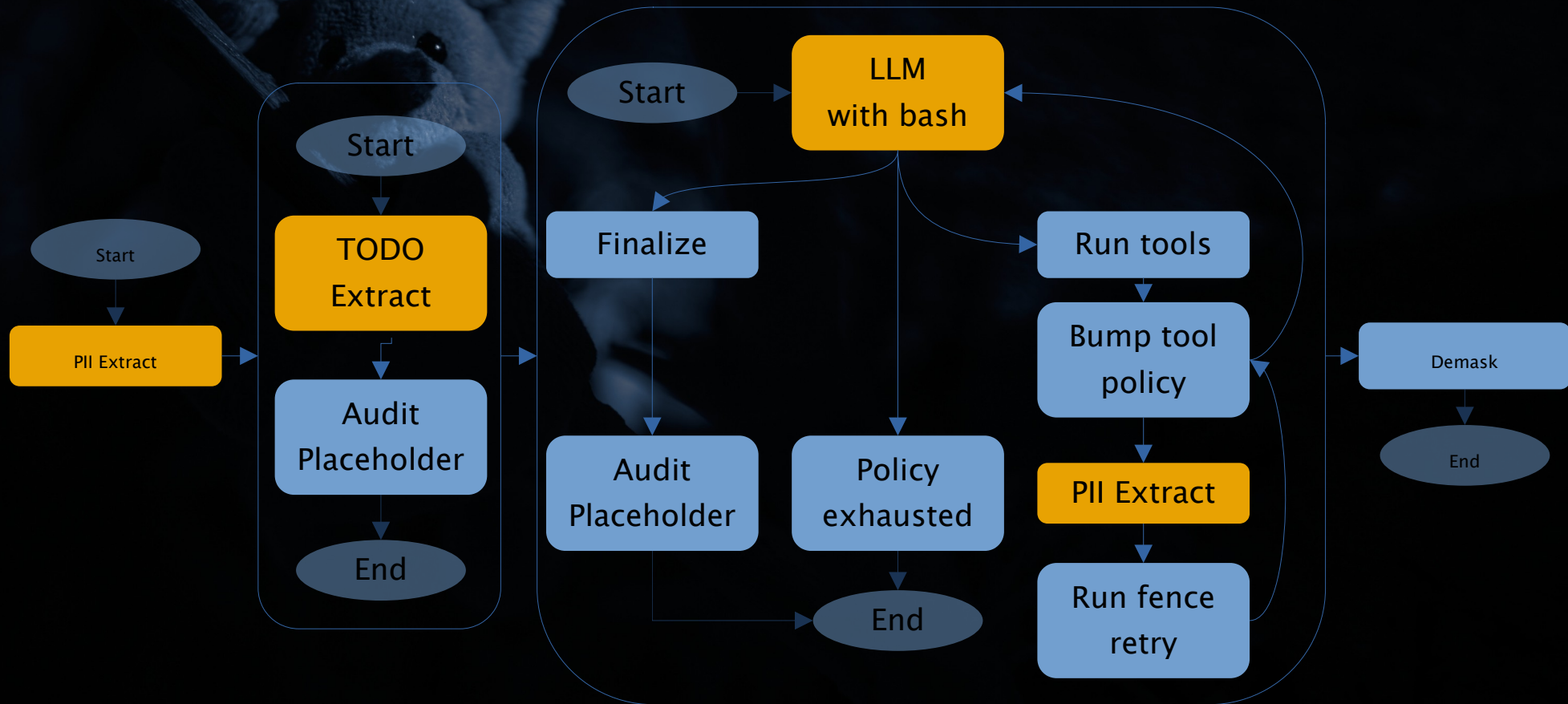
# The speaker says...

The **task seems trivial**. A frontier model could carry out the task in a single prompt. Yet the **scope is further reduced**:

- Only email addresses are considered PII
- Email addresses are the „who“, the assignee
- A TODO is a list of „who“ does „what“ by „when“

The scope reduction allows focus on the „mechanics“. There are challenges to explore regardless of the task size. For example, the previously raised question: **what are the boundaries for data, state and trust?**

# The graphs



# The speaker says...

Implementing **boundaries for data, state and trust** resulted in a graph:

- Using sequential execution at the top level
- Using plain nodes and **subgraphs**
- Having **few LLM components** (golden)
- Having **many deterministic Python components**

Zooming out and in will explain why. Next, we briefly zoom out to get an overview and look at the problem space. Then, we zoom in to some of the challenges.

# Problem space explored in the Lab

Non-deterministic LLM output,  
Typed state / schema friction, PII detection,  
Data minimization, Need-to-know subgraphs,  
Deterministic guards, Demasking boundary,  
Tracing / span tree, Chaos channels,  
Tool use, Sandboxing,  
Prompt drift,  
Symbolic tests, Mocked runs, E2E smoke eval

# The speaker says...

The word cloud sketches some of the challenges explored in the Lab experiment, using the example of a tiny task.

Those in blue are out of scope for this presentation, the ones in white are the ones we will zoom in on briefly. Please mind, this is a beginner's course: we scratch the surface.



# Zoom in: Masking PII

ulf@ example dot com  
to do something

E0\_afg32ahb  
to do something

PII Extract

mask\_pii\_email()

span: ulf@ example dot com  
raw: ulf@example.com

# The speaker says...

As per our scope definition only email addresses are considered personally identifiable information and need to be masked.

An email address can be written in many ways. Using both LLMs and Python code, the recognition task is carried out by the LLM. The LLM returns a JSON list with the occurrences. For each email address the text span in the input is provided together with what the LLM believes to be the normalized email. The Python function `mask_pii_email()` takes it from here: validate the raw email, remove all spans from the original text, double-check that no span was missed, replace the email with placeholder. Then return a list of placeholders and the masked text.

Ask yourself how to test this: LLM-as-a-judge? Golden Set? Established benchmarks? How to compare performance of specialized models? I didn't follow this up.

# Zoom in: State & Privacy

Global state:  
messages, pii\_email

Global state:  
messages, pii\_email,  
todo\_list

TODO state:  
masked\_text,  
placeholder\_allowlist,  
todo\_list



# The speaker says...

The PII extraction node uses GlobalState as its input. It needs to know the unmasked input.

The TODO list extraction LLM does not need the PII. Following our question **what are the boundaries for data, state and trust**, the TODO extraction node is a **subgraph**.

**Subgraphs are no magic** that solves all state and privacy challenges but they help to make data boundaries explicit:

- **Data minimization**: TODO list extraction can happen on placeholders instead of PII
- **Need-to-know subgraphs**: Bridges isolate state. The bridge ensures that the LLM does not have access to previous messages, and operates on the masked input text only.

Overkill? Depends on your data privacy rules!

# Zoom in: Deterministic guards

PII Extract

Spans missed?

...

TODO Extract

Valid schema?

Valid placeholder?

Demask

# The speaker says...

LLMs are not perfect: there are errors, there are hallucinations, answers are not deterministic.

**Deterministic guards** ensure boundaries. Here:

- **Typed state/schema friction**: using Pydantic to implement state objects
- **Privacy**: obvious PII leaks?
- **Consistency**: are placeholders known?
- **Demasking boundary**: trusted, deterministic code path

Subgraphs reduce what a node can see. Guards check what the node gives back.



# Zoom in: Reducer, traces, spans

Lab context manager for LangGraph reducer:

- Message transformation and monitoring

LiteLLM:

- Per call trace

Arize Phoenix:

- Spans – a tree-structured trace of multiple LLM calls

# The speaker says...

With boundaries and guards in place how can we **see what actually happened?**

The **Lab** offers **three ways to audit** the data exchanged with LLMs.

**LangGraph** offers a callback, the **reducer**, when messages are to be added to a graph's state. The reducer can **monitor messages but also transform them**, for example, to implement context compression or chaos testing. The Lab provides a Python context manager to set a custom reducer.

**LiteLLM**, the OpenAI API gateway, records messages at the **granularity of each LLM call**.

**Arize Phoenix** adds spans: a **tree view of a LangGraph graph invocation** with timings for all steps and data exchanged with the LLM, if any.

# Zoom in: Tool sandboxing

Powerful tools are useful — and dangerous.

Tool–Thorsten requests a bash script:

- HTTP bridge
- Sysbox / Docker-in-Docker container stdout / stderr
- Cleanup ...
- plus: the Lab with the agent runs in a virtual machine

# The speaker says...

Tool–Thorsten is the over–motivated agent that wants to try all the tools out there. He likes bash, just as coding agents like to use Python as a tool. Prompt engineering is not enough at this point. Tool usage is now (also) a systems problem: execution environment, isolation, timeouts, logging, cleanup:

- **Tool sandboxing:** sysbox (Docker–in–Docker) – **not production security [1]**
- Isolation: HTTP bridge to run bash in container, namespaces/shared kernel, session
- Timeout: best–effort attempt made to kill bash
- Logging: bash execution artifacts, Cleanup: best effort using finally

[1] Recently OpenAI acquired Ona, a Kiel–based startup, that has developed a production–grade solution to run background agents securely in the cloud. In comparison, Tool–Thorsten is not even a toy version.

# Learnings

Tiny task – big engineering surface:

- Tiny local models: great to explore limits
- State boundaries matter
- Deterministic guards matter
- Tracing helps debug agents
- ...

# The speaker says...

**Secure by default is today's standard.** Plan for it from the beginning. Prompt engineering does not provide security. Especially – but not only – small open-source models (3B/7B) need deterministic checks [1]. Observability at the granularity of each state transition is key to debugging. Auditability is a must-have for regulatory compliance.

The Toolbert Lab is a home for agents – well, it scratches the surface of the matter. In this home kids can focus on developing new skills without hurting themselves too much. Parenting is assisted by infrastructure tooling.

By **separating agent and infrastructure layers** both the kids and the house can grow independently towards whatever future interests there may be. On the agent side: RAG, context compression, ... , or on the infrastructure side: CI/CD, evals, monitoring, scalability, ...

[1] Large models bring new risks such as [AI Sandbagging](#)





The End



# The speaker says...

Thank you for listening!

Don't forget to check out <https://www.opencampus.sh/> for more courses & events.

# Extra: Pacing

Free-tier offerings limit requests per minute.

LiteLLM gateway:

- Default rate limiter: 429 Too Many Requests
- Mitigation: `async_pre_call_hook` plugin, `asyncio.sleep()`
- No agent code changes: the agent runs fine – but slow...

# The speaker says...

Still here? OK – a few extra slides for questions...

You probably hit the free-tier model limits, particularly the requests per minute. As a human you may be fine with something as low as two chat interactions per minute (2 rpm) to give a model a try.

But your agent's tool loop can easily go past 2 rpm even without any parallel execution and only a single agent acting. Unfortunately the default LiteLLM rate limiter throws 429 and does not slow down requests. Adding waits to your agent code is certainly the wrong layer.

Solution without agent code changes: LiteLLM plugin using the `async_pre_call_hook`.

# Extra: Container diagram

Developer

Cloud LLM

Dev

Python & Jupyter

Caddy

TLS reverse  
proxy

LiteLLM  
clean

OpenAI gateway

LiteLLM  
chaos

OpenAI gateway

Ollama

Local inference

Sysbox  
bash

Sandbox API

Postgres  
pgvector

Database

Phoenix

Observability

Toxiproxy

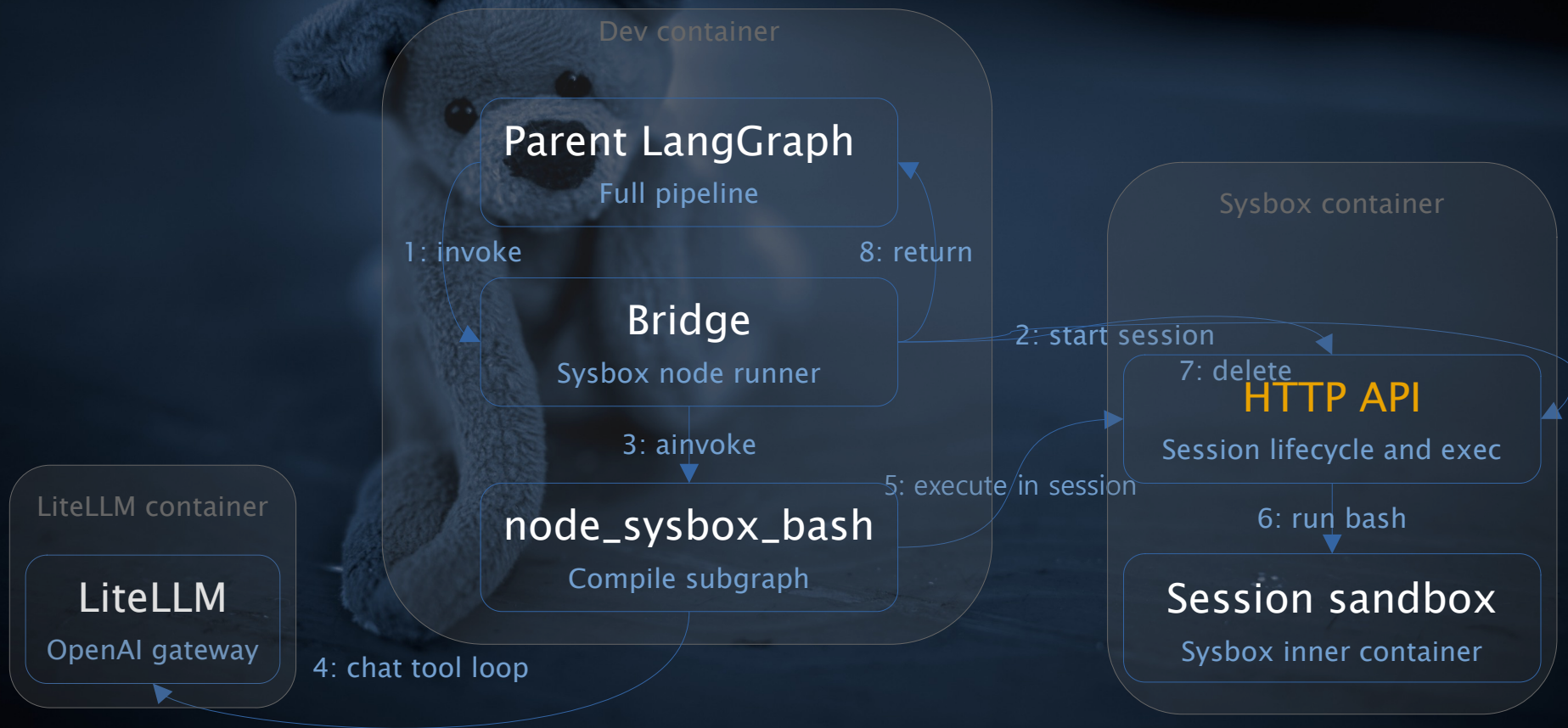
Chaos proxy

# The speaker says...

The slide may give you an idea of the size of the Lab.

I was curious about the home, and wanted something I can reuse for future experiments.

# Extra: Sysbox bash collaboration





# The speaker says...

The most basic sandboxing mistakes when working with agents include:

- Leaking secrets from the host filesystem (\$HOME, .aws, .env, .ssh, ...)
- Unlimited resources, unlimited network access, no timeouts
- When using containers: access to the host Docker socket

Containers can be used to mitigate those risks, and are convenient to use in a controlled Lab environment. The use of an HTTP API:

- Prevents access to the inner dockerd
- Establishes a controlled system boundary

# Extra: Lab sandboxing

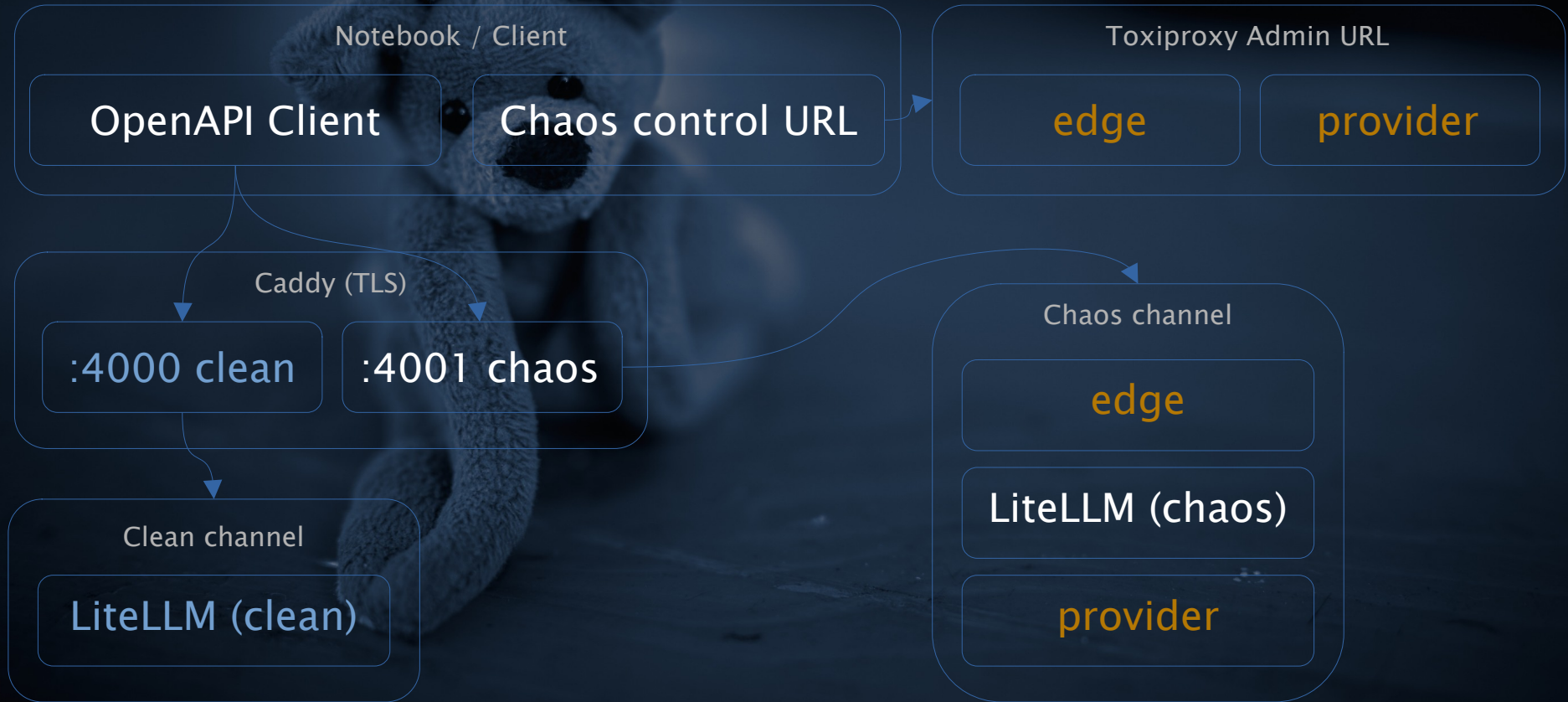
| Layer | Component        | Boundary                       | Policy                                           | Lifecycle                   |
|-------|------------------|--------------------------------|--------------------------------------------------|-----------------------------|
| L1    | Virtual Machine  | Hypervisor                     | vCPU/RAM/disk/net/<br>devices                    | Persistent guest FS         |
| L2    | dockerd          | shared kernel + namespaces     | runtime selection, caps,<br>cgroups, mounts, net | daemon, images,<br>volumes, |
| L3    | sysbox_runc      | usersns + virtualized proc/sys | UID map; reduced host<br>view                    | compose service             |
| L4    | Sysbox container | outer agent sandbox            | HTTP API only; resource<br>limits                | per-agent service           |
| L5    | Inner container  | inner workload sandbox         | non-root; timeout;<br>FS/net policy              | per-command/<br>session     |

# The speaker says...

Sysbox does not provide the same level of isolation as a virtual machine because of its larger attack face, and that matters; see also „Quantifying Frontier LLM Capabilities for Container Sandbox Escape“ .

Luis Cardoso wrote an excellent blog post on the topic of sandboxes: A field guide to sandboxes for AI.

# Extra: Chaos container diagram



# The speaker says...

For chaos testing, OpenAPI clients either connect to a clean or a chaos endpoint. On the chaos channel, clients can configure Toxiproxy toxics for any of the two proxies:

- edge: for testing client network issues
- provider: for testing LiteLLM retry and fallback mechanisms